Setting Up a Django Project with MongoDB Atlas and AI Recommender Integration with Render Deployment

Intended Learning Outcomes

At the end of this activity, you should be able to:

1. Set up a Django project and configure it to work with MongoDB using **djongo**.
2. Create a Django app and connect it to a MongoDB Atlas cluster for data storage.
3. Define models, views, and templates to handle product data in an e-commerce website.
4. Implement an AI-based recommender system to enhance user experience with personalized product suggestions.
5. Deploy the Django project to Render, integrating version control with GitHub for project management.

Objectives:

By completing this activity, you will:

1. Learn how to set up a Python virtual environment and install necessary dependencies for Django and MongoDB.
2. Understand how to define models in Django and connect them to a MongoDB database for storage and retrieval.
3. Practice implementing basic views, URLs, and templates in Django to manage and display products.
4. Integrate an AI-based recommender system to suggest relevant products based on user behavior or data.
5. Gain hands-on experience with version control using Git and GitHub, and deploy a Django project to Render for live production.

Instructions: Before we start, Make sure you have:

- Python installed (3.8 to Version 3.10.0)
- MongoDB Atlas account (free tier is fine)
- `pip` or `pipenv`
- Virtualenv (`pip install virtualenv`)

Now let's create a project folder and virtual environment

```
PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic> mkdir AI_Recommender_Ecommerce

    Directory: C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        07/04/2025  12:08 pm                AI_Recommender_Ecommerce

PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic> cd AI_Recommender_Ecommerce
PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce> virtualenv venv
```

```
PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce> venv/Scripts/activate
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce>
```

Install **Djongo** and **MongoDB** support

```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce> pip install django djongo pymongo
Collecting django
  Downloading Django-5.2-py3-none-any.whl.metadata (4.1 kB)
Collecting djongo
  Downloading djongo-1.3.7-py3-none-any.whl.metadata (1.8 kB)
Collecting pymongo
  Downloading pymongo-4.11.3-cp313-cp313-win_amd64.whl.metadata (22 kB)
Collecting asgiref>=3.8.1 (from django)
  Using cached asgiref-3.8.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.3.1 (from django)
  Using cached sqlparse-0.5.3-py3-none-any.whl.metadata (3.9 kB)
```

djongo *is a Django connector that allows working with* ***MongoDB*** *as if it were a relational DB*

Let's start a Django project

```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce> django-admin startproject ecommerce_project
```

Create an app called **`store`**

```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce> cd ecommerce_project
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project> python manage.py startapp store
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project>
```

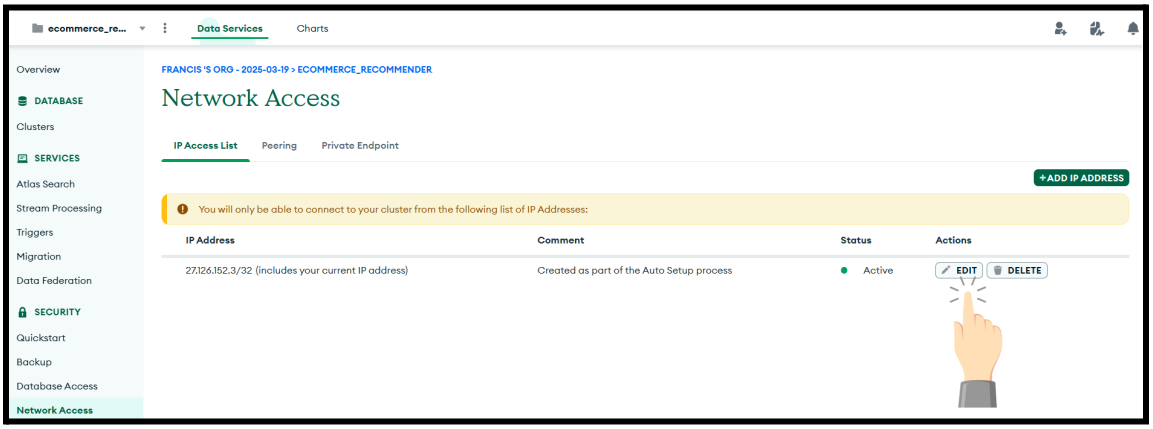
Create a MongoDB Atlas Cluster

Create a free cluster (default name is usually Cluster0). That's fine!

Click "Database Access" → Add a user: example:

- Username: storeadmin
- Password: admin12345 (or your own)

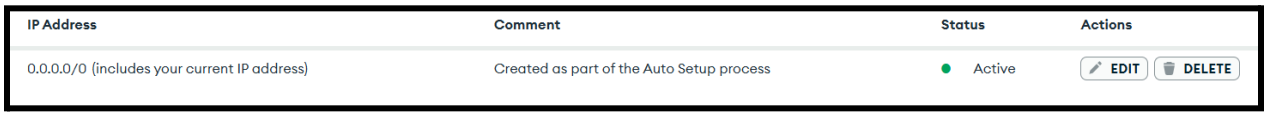
Click "Network Access" → Add Edit IP address:



- Click Edit and Use **"Allow Access from Anywhere"**
- If you click "EDIT" and select "Allow access from anywhere," it will automatically add: **0.0.0.0/0**

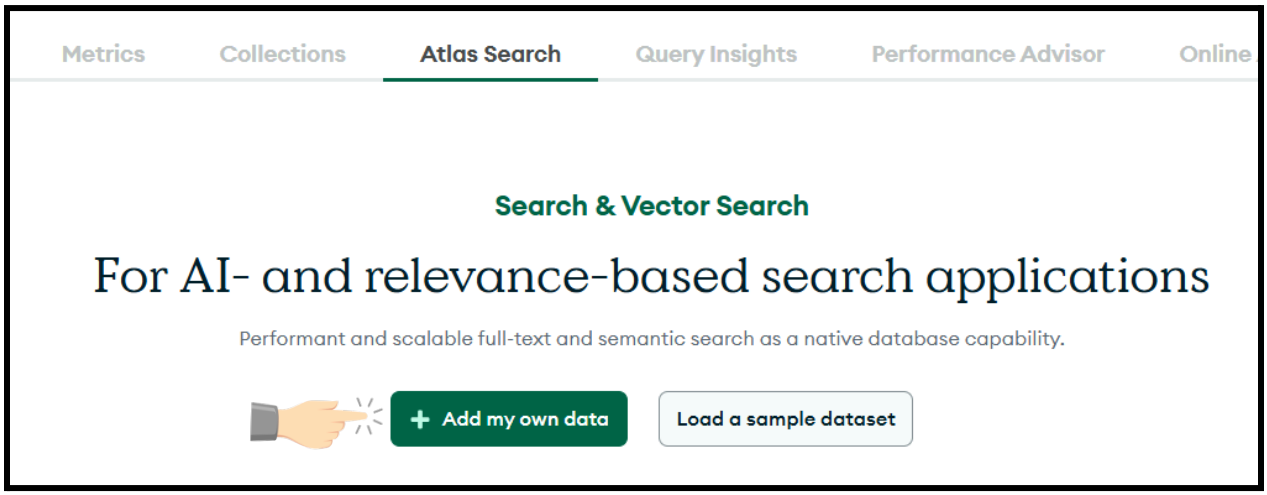
This allows any device to connect to the cluster — useful if you're:

- **Developing on multiple devices (home, school, laptop, etc.**
- **Working with a team**
- **Avoiding issues with changing IPs**

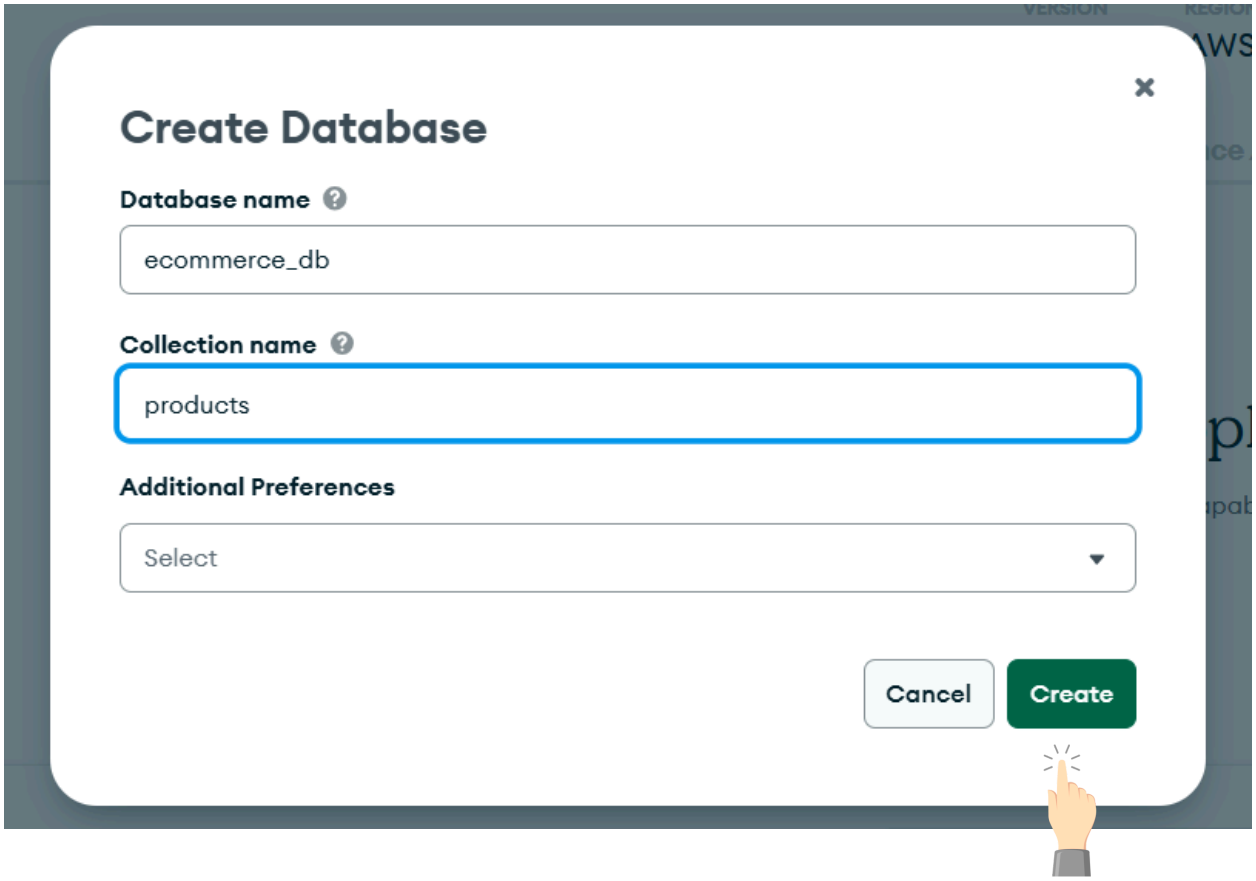


Click "Atlas Search" tab → "Atlas Search"

- Click Add My Own Data

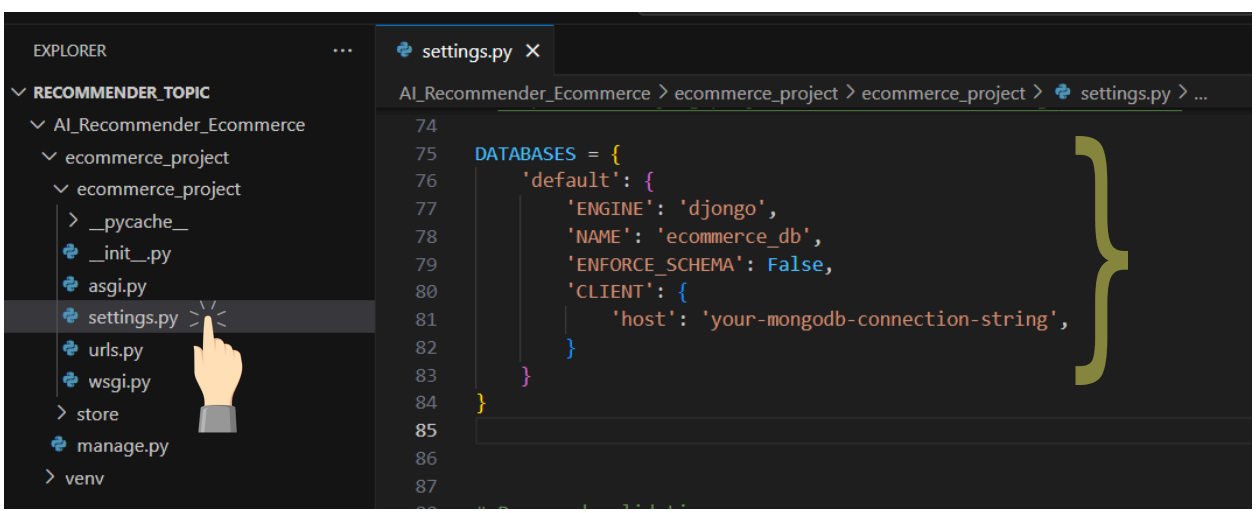


- **Database Name**: ecommerce_db
- **Collection Name**: products

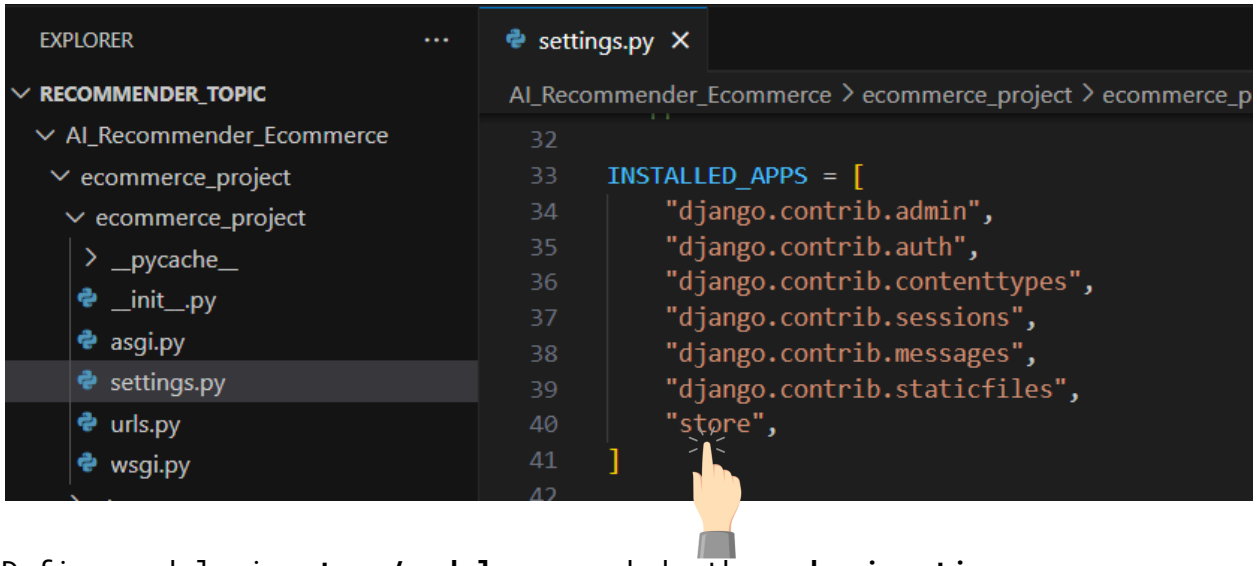


Step 6: Update Django settings.py

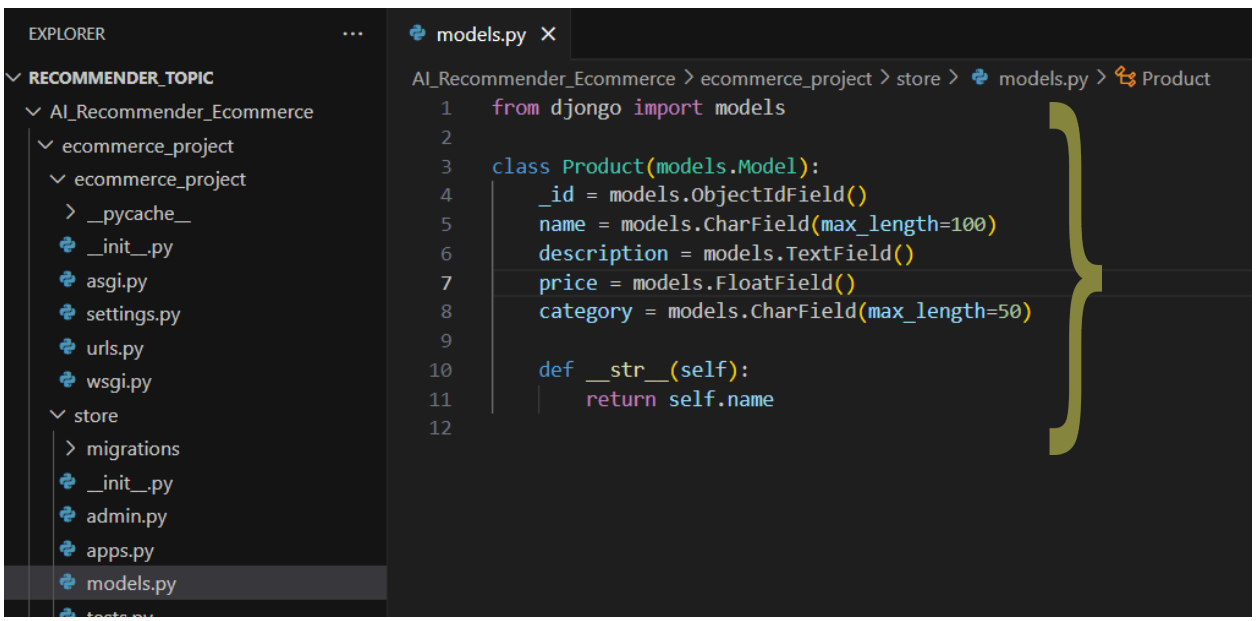
In **ecommerce_project/settings.py**, update the **DATABASES** block and add your **mongodb string**:



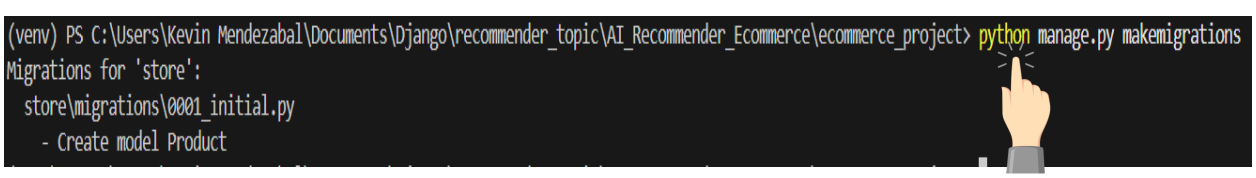
Also add **'store'** to your **INSTALLED_APPS** list.



Define models in **store/models.py** and do the **makemigrations**



Note: No migrations are needed for **djongo**. But you can still use **makemigrations** if desired.



```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, store
Running migrations:
This version of djongo does not support "NULL, NOT NULL column validation check" fully. Visit https://nesdis.github.io/djongo/support/
  Applying contenttypes.0001_initial...This version of djongo does not support "schema validation using CONSTRAINT" fully. Visit https://nesdis.gi
ub.io/djongo/support/
 OK
  Applying auth.0001_initial...This version of djongo does not support "schema validation using KEY" fully. Visit https://nesdis.github.io/djongo/s
pport/
/support/
```

Add basic views in **store/views.py**

```
EXPLORER
RECOMMENDER_TOPIC
  AI_Recommender_Ecommerce
    ecommerce_project
      ecommerce_project
      store
        migrations
        __init__.py
        admin.py
        apps.py
        models.py
        tests.py
        views.py
      manage.py
```

models.py | views.py

AI_Recommender_Ecommerce > ecommerce_project > store > views.py > ...

```python
from django.shortcuts import render
from .models import Product

def product_list(request):
    products = Product.objects.all()
    return render(request, 'store/product_list.html', {'products': products})

```

Create the file **store/urls.py:**

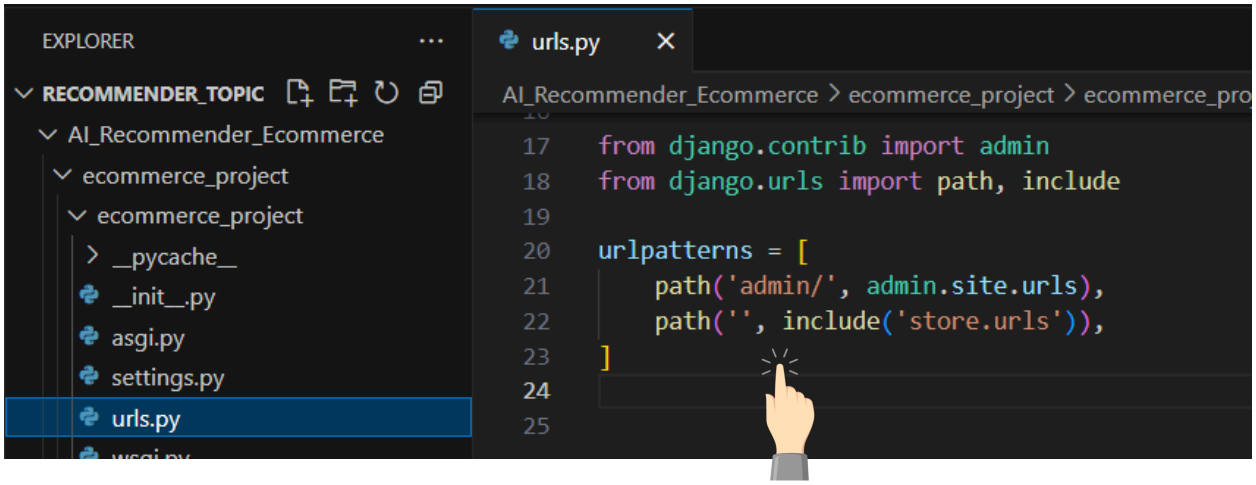
```
EXPLORER
RECOMMENDER_TOPIC
  AI_Recommender_Ecommerce
    ecommerce_project
      ecommerce_project
      store
        migrations
        __init__.py
        admin.py
        apps.py
        models.py
        tests.py
        urls.py
        views.py
```

models.py | views.py | urls.py

AI_Recommender_Ecommerce > ecommerce_project > store > urls.py > ...

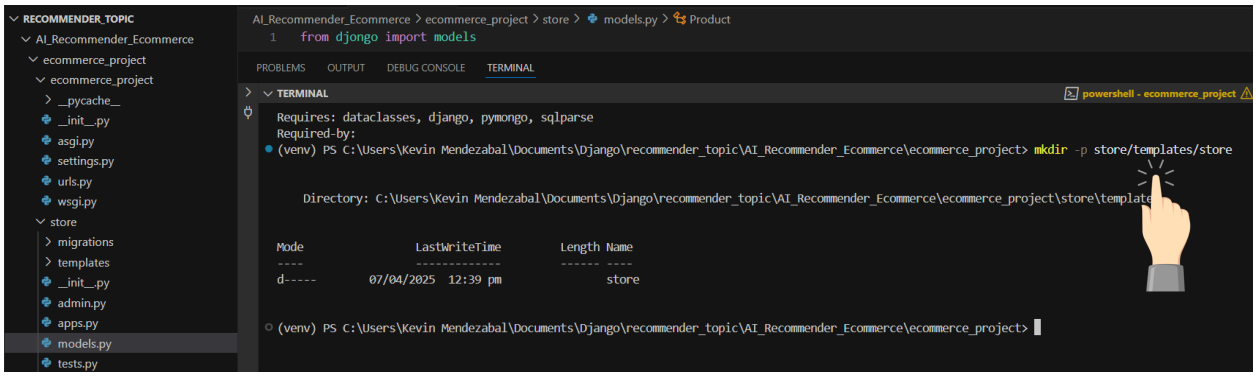
```python
from django.urls import path
from . import views

urlpatterns = [
    path('', views.product_list, name='product_list'),
]

```

Connect app URLs in **ecommerce_project/urls.py**

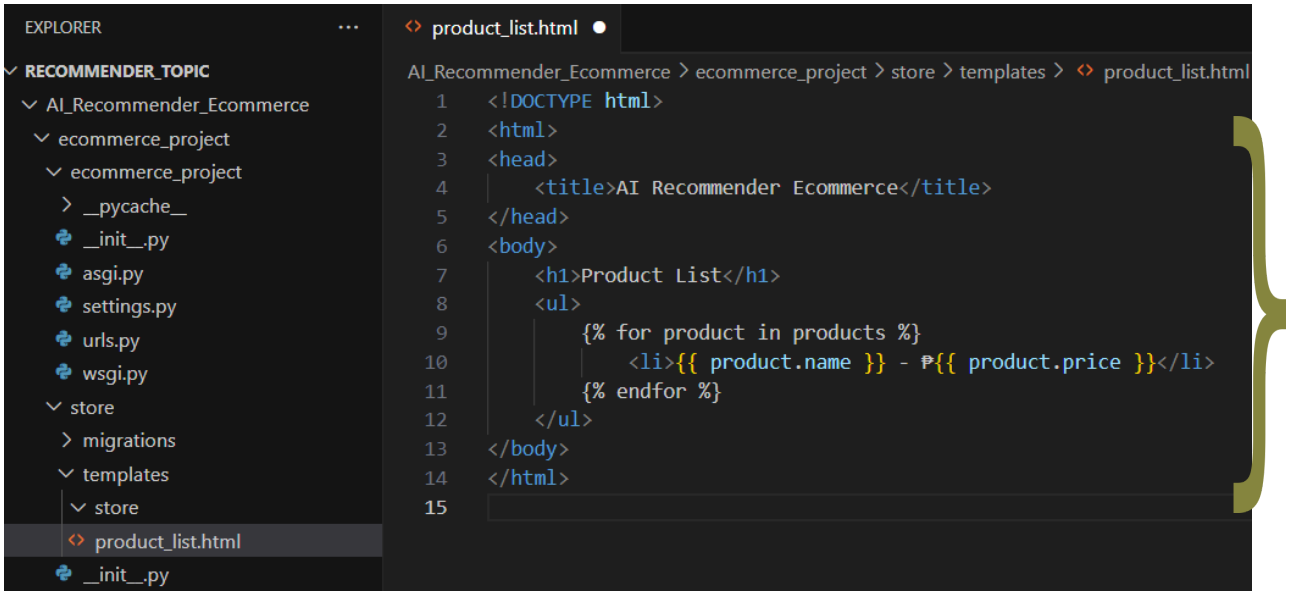


Setup template structure

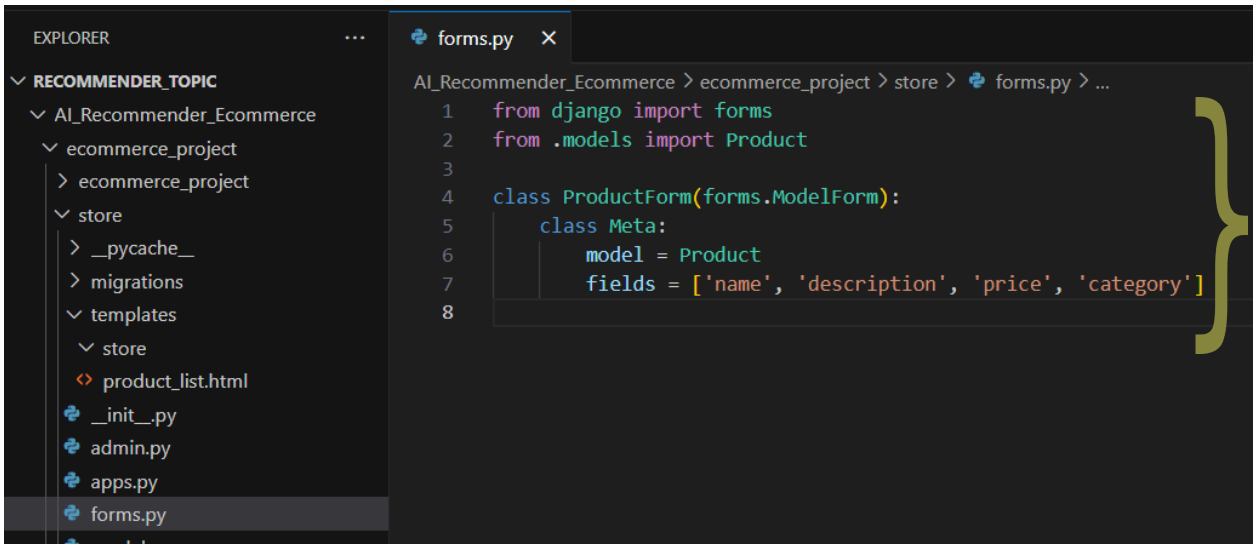
Create folders: **mkdir -p store/templates/store**



Create **store/templates/store/product_list.html:**



Inside your store app, create a file called forms.py



Run the Server

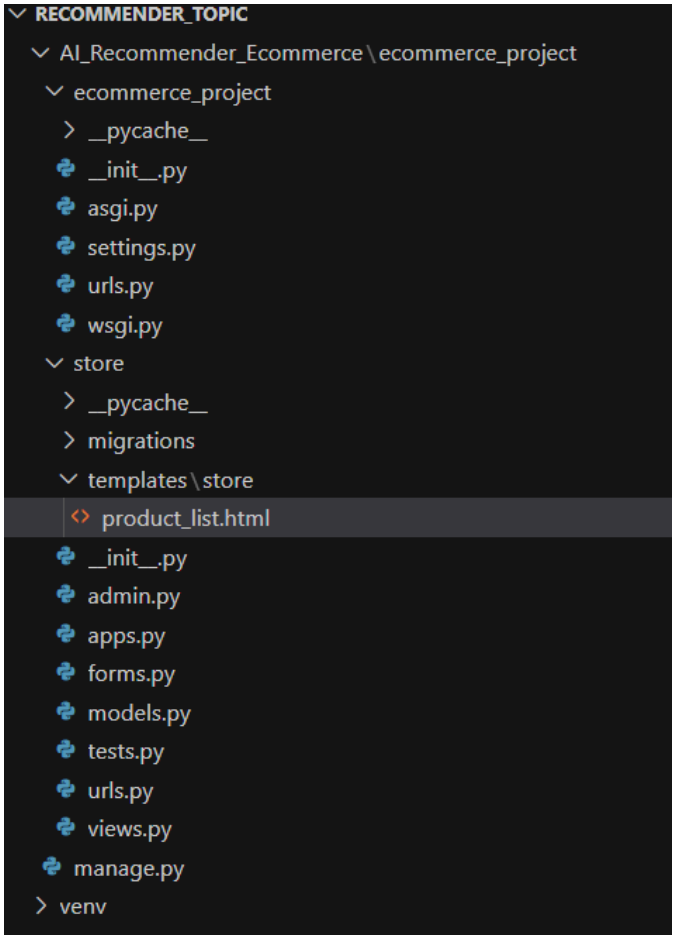
python manage.py runserver
(if there's an error such as ModuleNoFoundError: No Module found pytz, then install it as **pip install pytz**,

```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project> pip install pytz
Collecting pytz
  Using cached pytz-2025.2-py2.py3-none-any.whl.metadata (22 kB)
Using cached pytz-2025.2-py2.py3-none-any.whl (509 kB)
Installing collected packages: pytz
Successfully installed pytz-2025.2
```

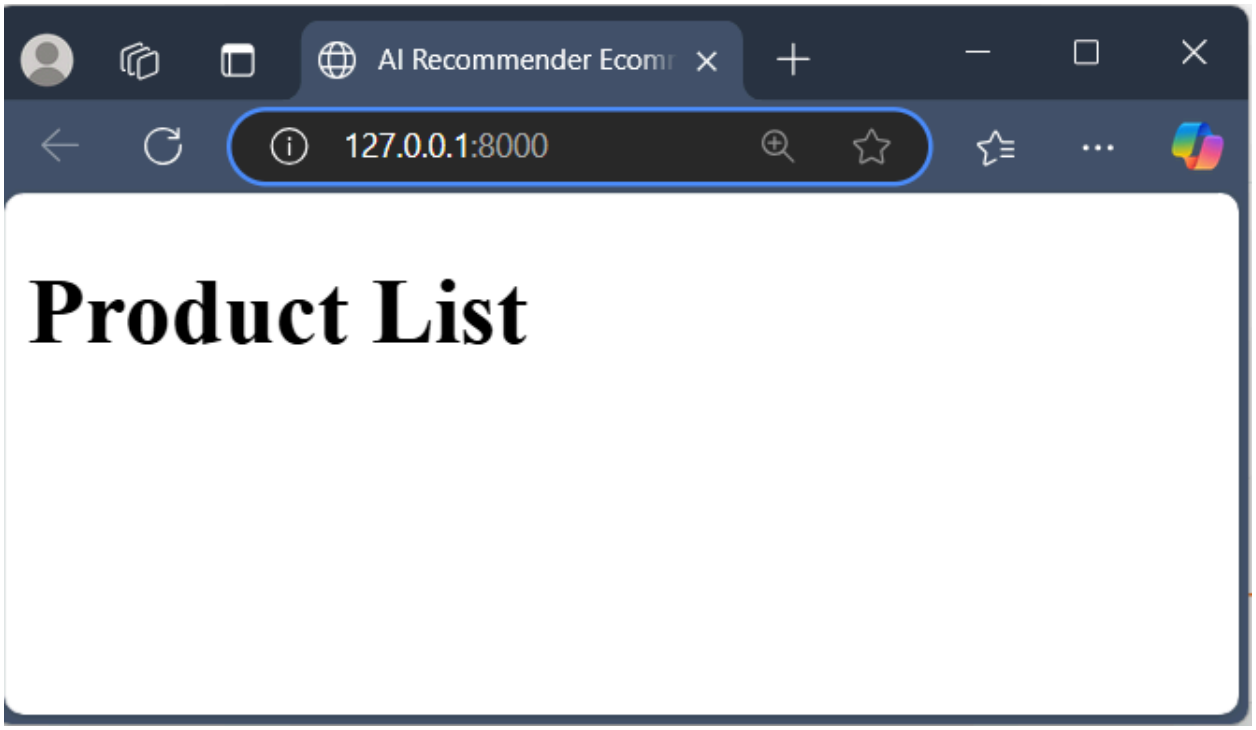
and if the error was the djongo is improperlyConfigured, then verify if your djongo is compatible with your python version, it must be recommended to Downgrade your python 11 to 13 into **Python 3.10** or **3.9**, which are better supported by Djongo and MongoDB drivers. Upon Installation, you need to install the following:

1. Virtualenv
2. dnspython

If error persist, kindly check your folder Structure:



Now runserver and Go to: **http://127.0.0.1:8000**



Let's Add a Simple AI-Based Recommender

In **store/views.py**, add a view to display and handle the product form:

views.py

AI_Recommender_Ecommerce > ecommerce_project > store > views.py > ...

```python
from django.shortcuts import render, redirect
from .forms import ProductForm
from .models import Product

def product_list(request):
    products = Product.objects.all()
    return render(request, 'store/product_list.html', {'products': products})

def add_product(request):
    if request.method == 'POST':
        form = ProductForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('product_list')  # change to your product list route name
    else:
        form = ProductForm()
    return render(request, 'store/add_product.html', {'form': form})
```

Create **add_product.html:**

```
<!DOCTYPE html>
<html>
<head>
    <title>Add Product</title>
</head>
<body>
    <h2>Add New Product</h2>

    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Add Product</button>
    </form>

    <a href="{% url 'product_list' %}">Back to Product List</a>
</body>
</html>
```

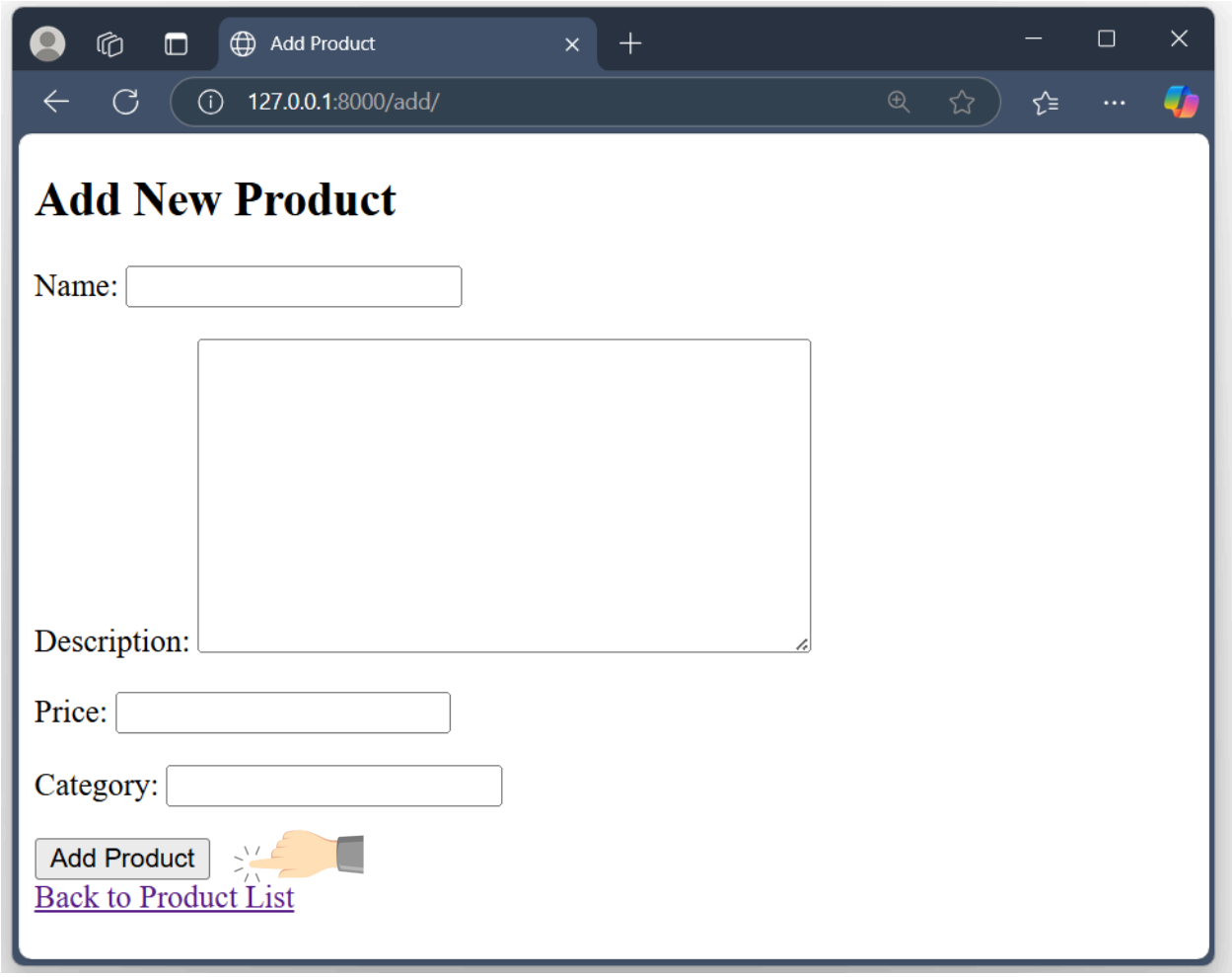
Update your **store/urls.py** to include the new view:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.product_list, name='product_list'),
    path('add/', views.add_product, name='add_product'), #add this
]
```

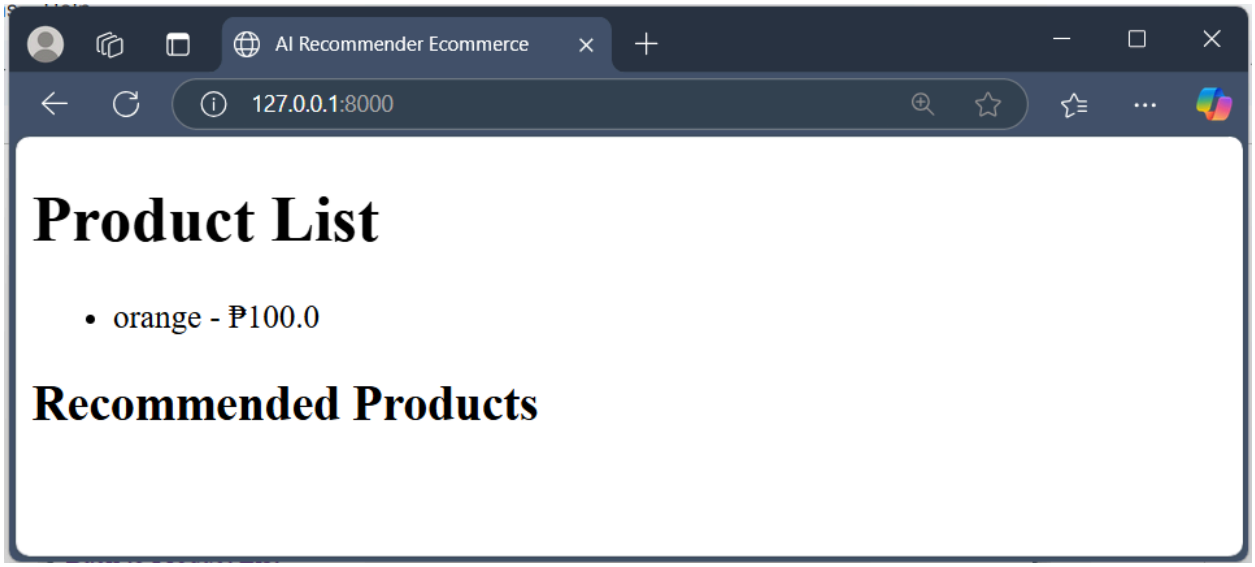
Also make sure your **ecommerce_project/urls.py** includes the store app's URLs:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('store.urls')), #this line
]
```

Run the server and go to: **http://127.0.0.1:8000/add/**



After fill-in the fields, click the **Add to Product Button**



You can also view all the products

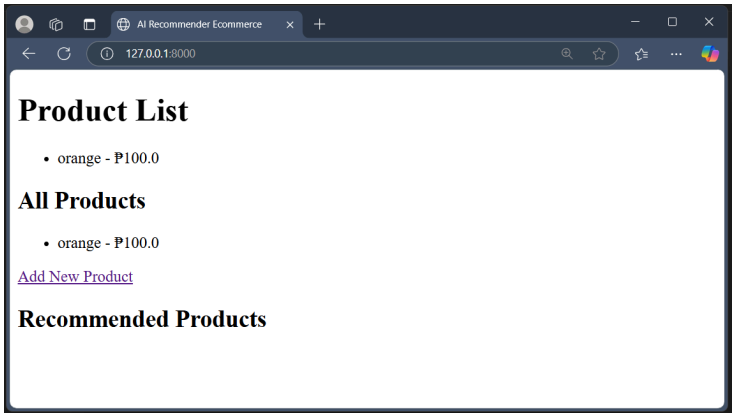
If not done yet, your **product_list** template could look like this:

views.py product_list.html

AI_Recommender_Ecommerce > ecommerce_project > store > templates > store > product_list.html > html > body

```html
<!DOCTYPE html>
<html>
<head>
    <title>AI Recommender Ecommerce</title>
</head>
<body>
    <h1>Product List</h1>
    <ul>
        {% for product in products %}
            <li>{{ product.name }} - ₱{{ product.price }}</li>
        {% endfor %}
    </ul>

    <h2>All Products</h2>
    <ul>
        {% for product in products %}
            <li>{{ product.name }} - ₱{{ product.price }}</li>
        {% endfor %}
    </ul>
    <a href="{% url 'add_product' %}">Add New Product</a>

        Chat (CTRL + I) / Edit (CTRL + L)
    <h2>Recommended Products</h2>
<ul>
    {% for product in recommendations %}
        <li>{{ product.name }} - ₱{{ product.price }}</li>
    {% endfor %}
</ul>

</body>
</html>
```

Reload your server and check the output:



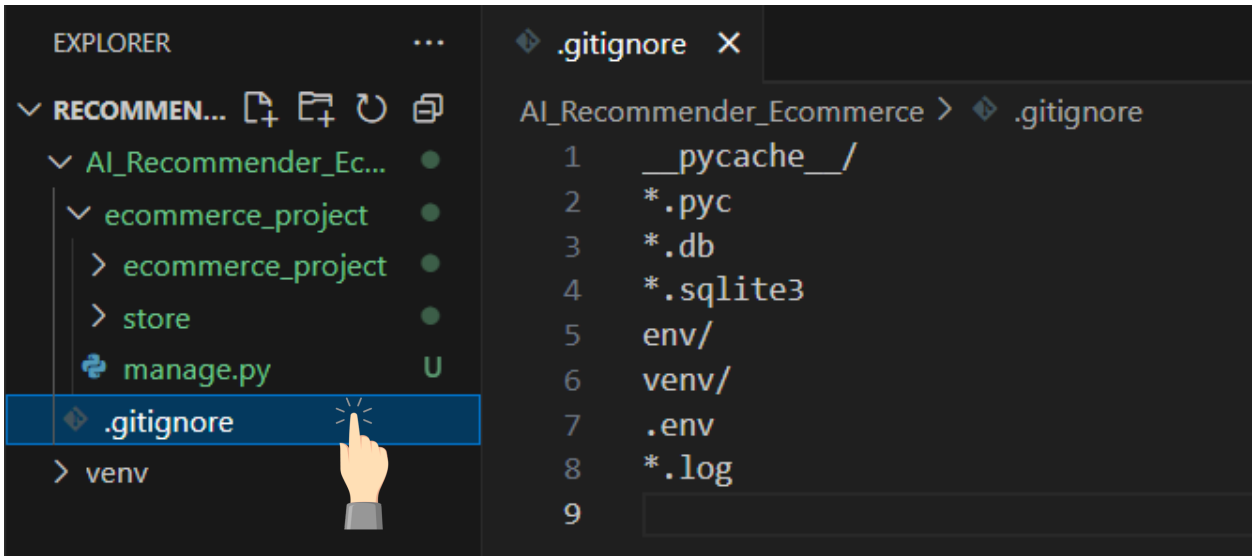
Final Steps: Version Control & Deployment

Push Your Django Project to GitHub

1. Initialize **Git**

```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project> git init
Initialized empty Git repository in C:/Users/Kevin Mendezabal/Documents/Django/recommender_topic/AI_Recommender_Ecommerce/ecommerce_project/.git/
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project>
```

2. Create a **.gitignore** file and Add this to ignore unnecessary files:



Now let's commit it

Add all files (except those ignored by .gitignore) and Make your first commit

```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project> code .gitignore
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project>
```

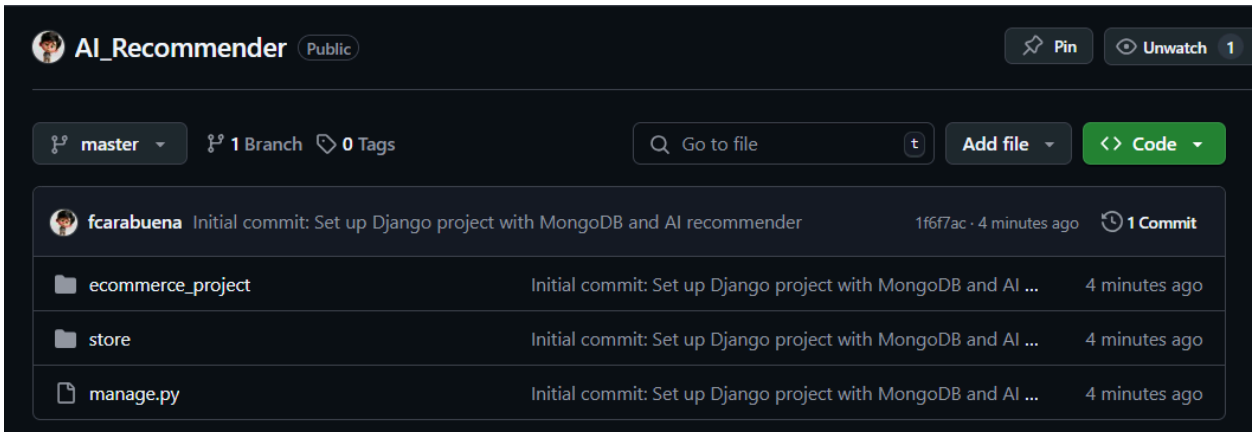
```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project> git add .
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project> git commit -m "Initial commit: Set up Django project with MongoDB and AI recommender"
[master (root-commit) 1f6f7ac] Initial commit: Set up Django project with MongoDB and AI recommender
 35 files changed, 333 insertions(+)
 create mode 100644 ecommerce_project/__init__.py
 create mode 100644 ecommerce_project/__pycache__/__init__.cpython-310.pyc
 create mode 100644 ecommerce_project/__pycache__/__init__.cpython-313.pyc
 create mode 100644 ecommerce_project/__pycache__/settings.cpython-310.pyc
 create mode 100644 ecommerce_project/__pycache__/settings.cpython-313.pyc
 create mode 100644 ecommerce_project/__pycache__/urls.cpython-310.pyc
 create mode 100644 ecommerce_project/__pycache__/urls.cpython-313.pyc
 create mode 100644 ecommerce_project/__pycache__/wsgi.cpython-310.pyc
 create mode 100644 ecommerce_project/asgi.py
 create mode 100644 ecommerce_project/settings.py
 create mode 100644 ecommerce_project/urls.py
 create mode 100644 ecommerce_project/wsgi.py
 create mode 100644 manage.py
 create mode 100644 store/__init__.py
 create mode 100644 store/__pycache__/__init__.cpython-310.pyc
 create mode 100644 store/__pycache__/__init__.cpython-313.pyc
```

link your local repo to GitHub:

```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project> git remote add origin https://github.com/fcarabuena/AI_Recommender.git
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project>
```

Then push your code:

```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project> git push -u origin master
Enumerating objects: 43, done.
Counting objects: 100% (43/43), done.
Delta compression using up to 4 threads
Compressing objects: 100% (41/41), done.
Writing objects: 100% (43/43), 12.35 KiB | 2.47 MiB/s, done.
Total 43 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), done.
To https://github.com/fcarabuena/AI_Recommender.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

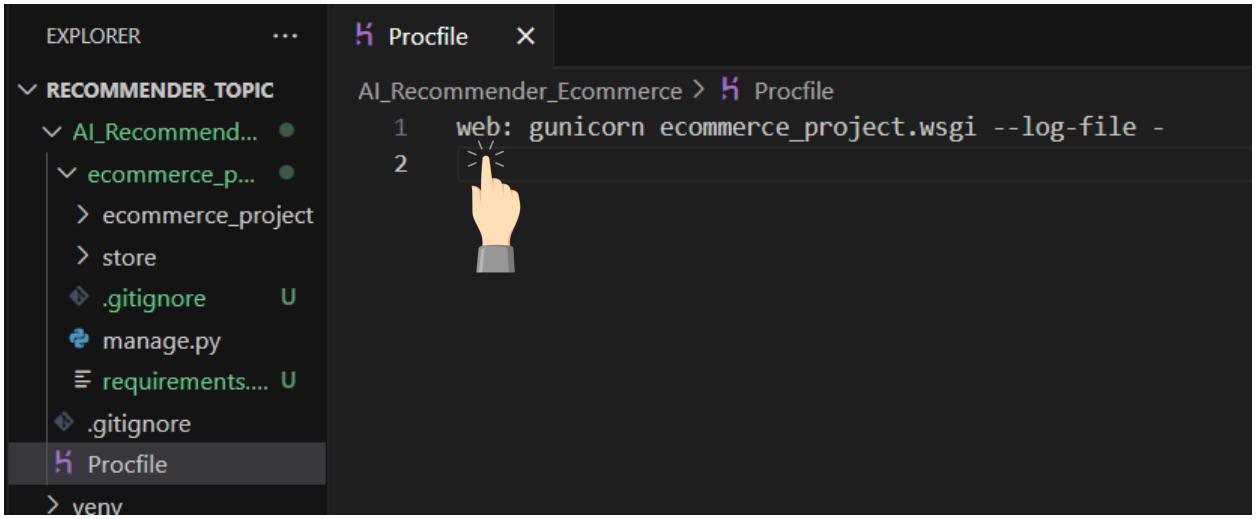


Deploying to Render (FREE Tier)

1: Prepare Files for Render: Create **requirements.txt**

```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project> pip freeze > requirements.txt
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project>
```

2. Create Procfile: This tells Render how to run your app:



In your virtual environment, run: pip install gunicorn

This is the server that Render will use to run your Django app in production.

```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project> pip install gunicorn
Collecting gunicorn
  Downloading gunicorn-23.0.0-py3-none-any.whl.metadata (4.4 kB)
Collecting packaging (from gunicorn)
  Downloading packaging-24.2-py3-none-any.whl.metadata (3.2 kB)
Downloading gunicorn-23.0.0-py3-none-any.whl (85 kB)
Downloading packaging-24.2-py3-none-any.whl (65 kB)
Installing collected packages: packaging, gunicorn
Successfully installed gunicorn-23.0.0 packaging-24.2
```

Update settings.py for production:

```
EXPLORER                                settings.py !M, M ×   COMMIT_EDITMSG   Procfile
RECOMMENDER_TOPIC                       AI_Recommender_Ecommerce > ecommerce_project > ecommerce_project > settings.py > ...
  AI_Recommend...                       SECRET_KEY = 'django-insecure-w+j5m5r_40j&km4&yn-summ1k1%1g95%t3wnc8k&2m%r014uyv
  ecommerce_p...
    ecommerce_...
      __pycache__
      __init__.py
      asgi.py
      settings...  !M, M
      urls.py
      wsgi.py
    store
    .gitignore
    manage.py
    requirements.txt
  .gitignore

24
25  # SECURITY WARNING: don't run with debug turned on in production!
26  DEBUG = True
27
28  import os
29
30  # Environment variables
31  SECRET_KEY = os.getenv('ggxlia&%!p6nyw9lh8kuwzdq@h-)vp#ey6@rjd2d2syr3%y#!2"', "fallback_dev_secret")
32  DEBUG = False
33  ALLOWED_HOSTS = ['ai-recommender-k6wy.onrender.com', 'localhost', '127.0.0.1']
34
35  # Add your environment variables for MongoDB and OpenAI      You, 34 minutes ago • Add requirements.t
36  OPENAI_KEY = os.getenv("dab82bcf11b83362167ef1634b859e6a")
37  MONGO_URI = os.getenv("d5f00a3894197fed3ab7df49986a603c")
38
```

To get your django secret key: Type this command in your terminal:

python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"

Thanks me later hehe :) yung fallback dev naman, for production na yan, and it's okay.

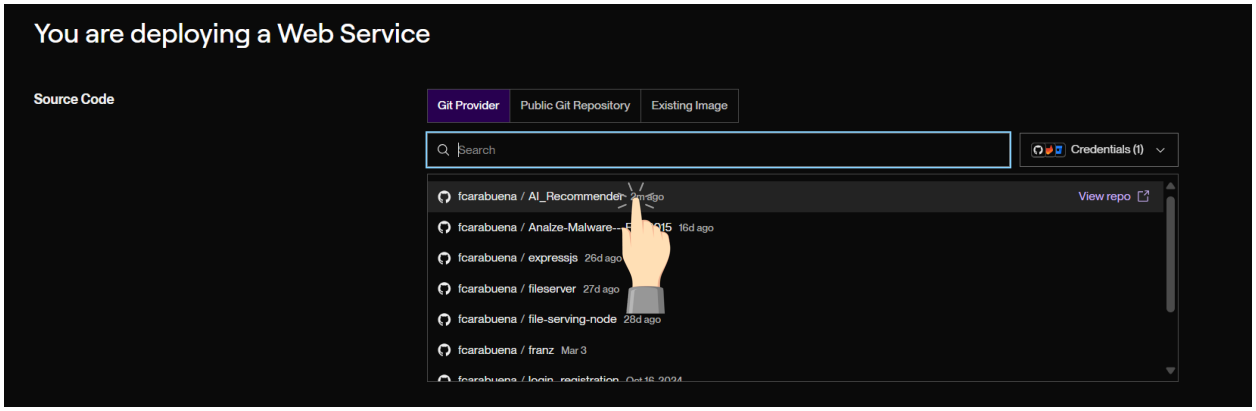
```
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project> python -c "from django.core.management.utils im
port get_random_secret_key; print(get_random_secret_key())"
ggxlia&%!p6nyw9lh8kuwzdq@h-)vp#ey6@rjd2d2syr3%y#!2
(venv) PS C:\Users\Kevin Mendezabal\Documents\Django\recommender_topic\AI_Recommender_Ecommerce\ecommerce_project>
```

Then commit changes from VSCode to your Github Repository.

Deploy on <u>Render</u>

1. Sign in or sign up at <u>https://render.com</u>
2. Click "**New Web Service**"

3. Connect your GitHub repo



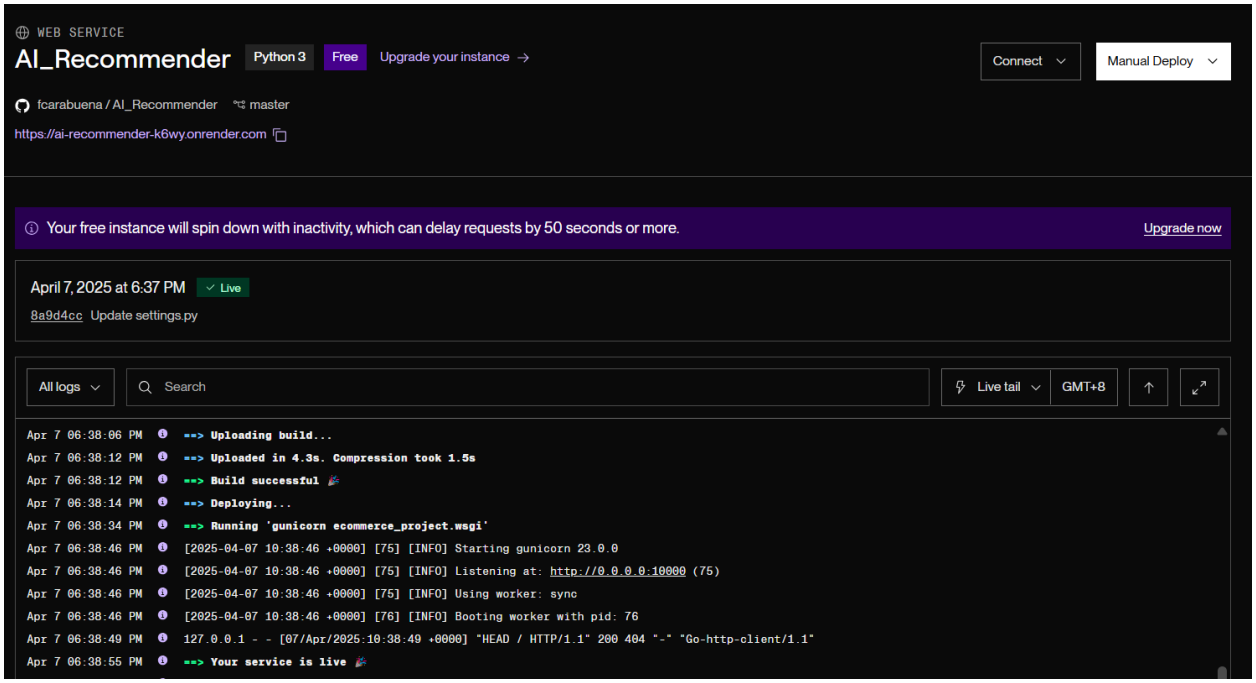
4. Set:

Build Command: pip install -r requirements.txt

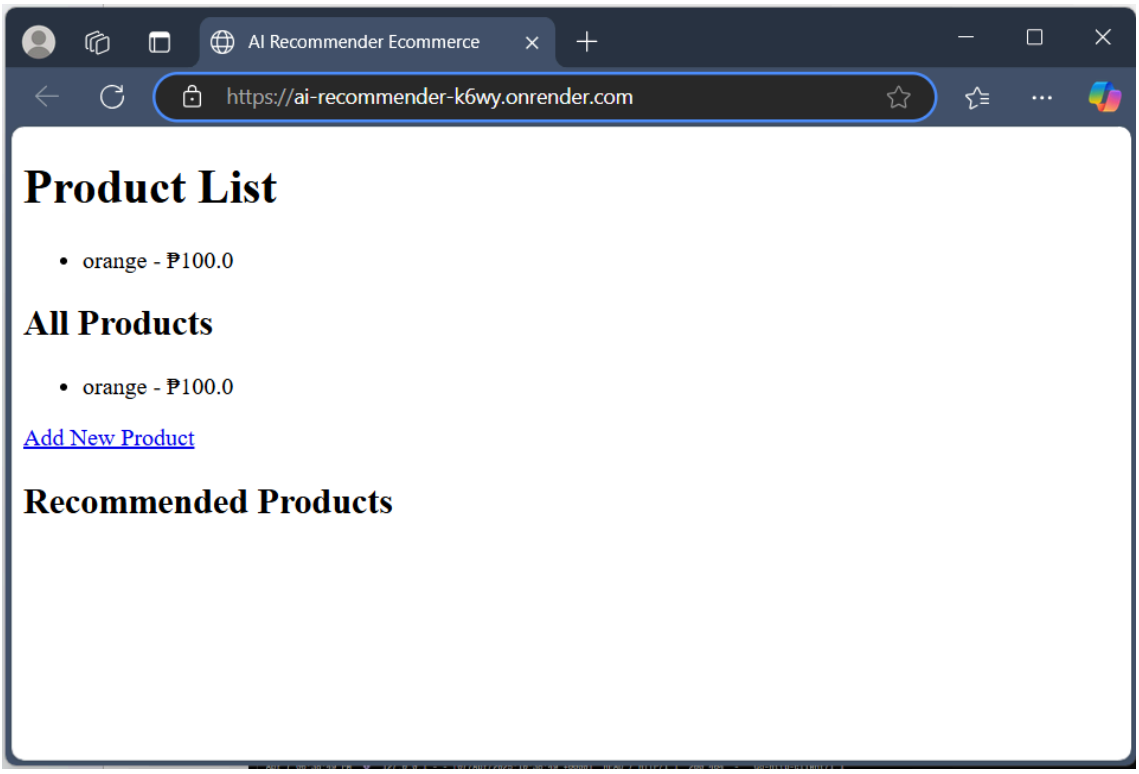
Start Command: gunicorn ecommerce_project.wsgi

Add environment variables from your .env file (OpenAI key, Django secret, Mongo URI)

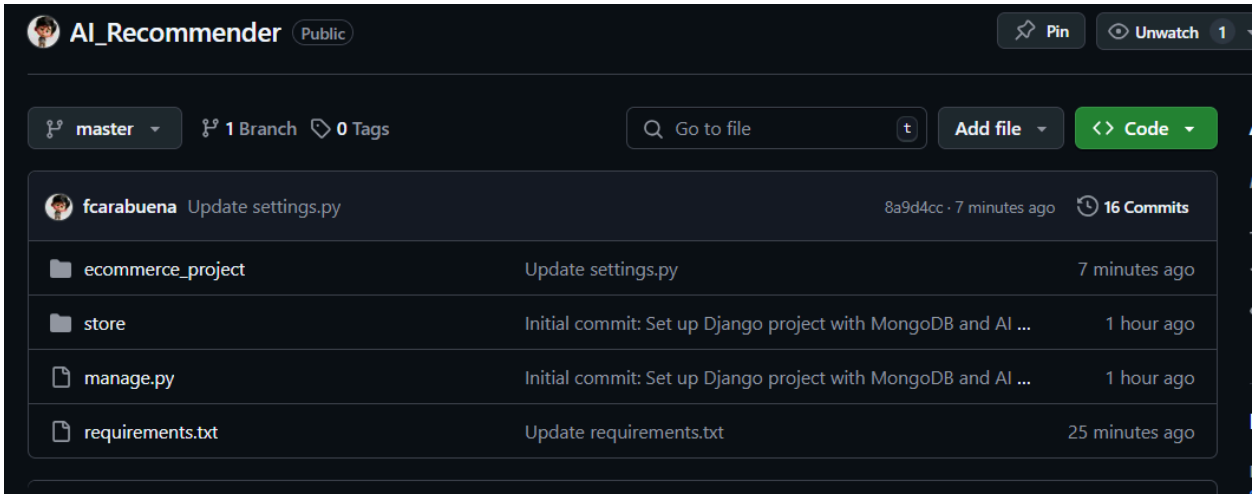
Render will install dependencies and deploy your app live.

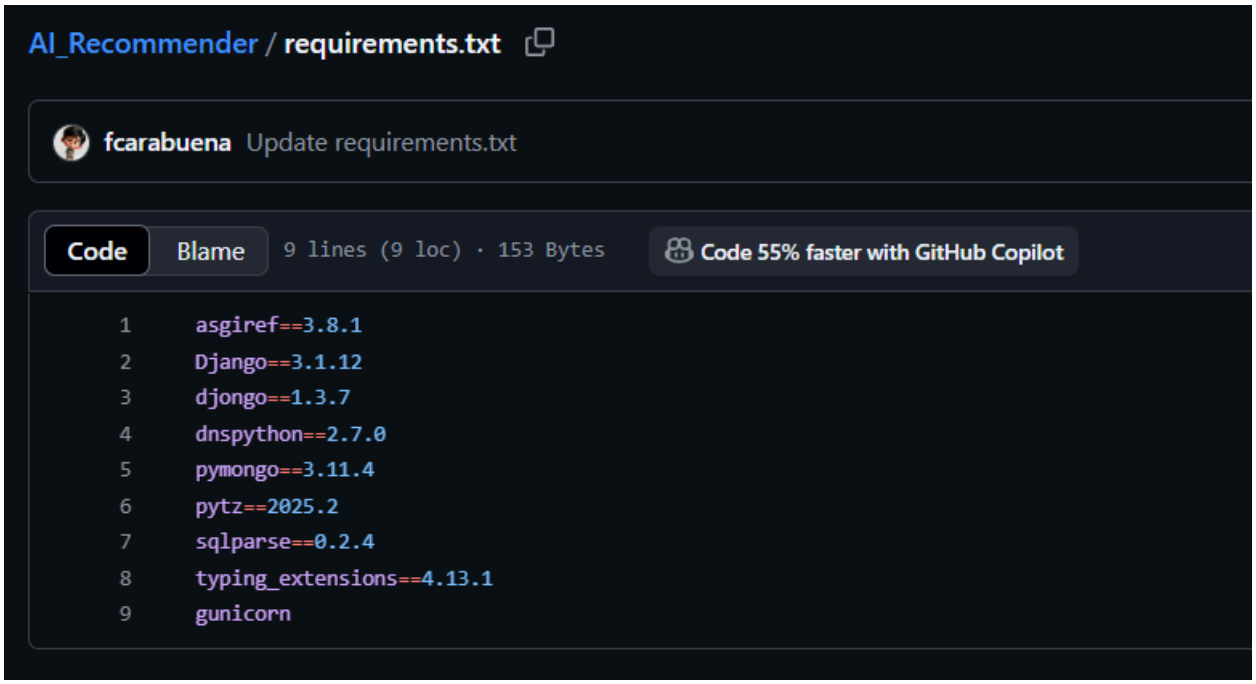


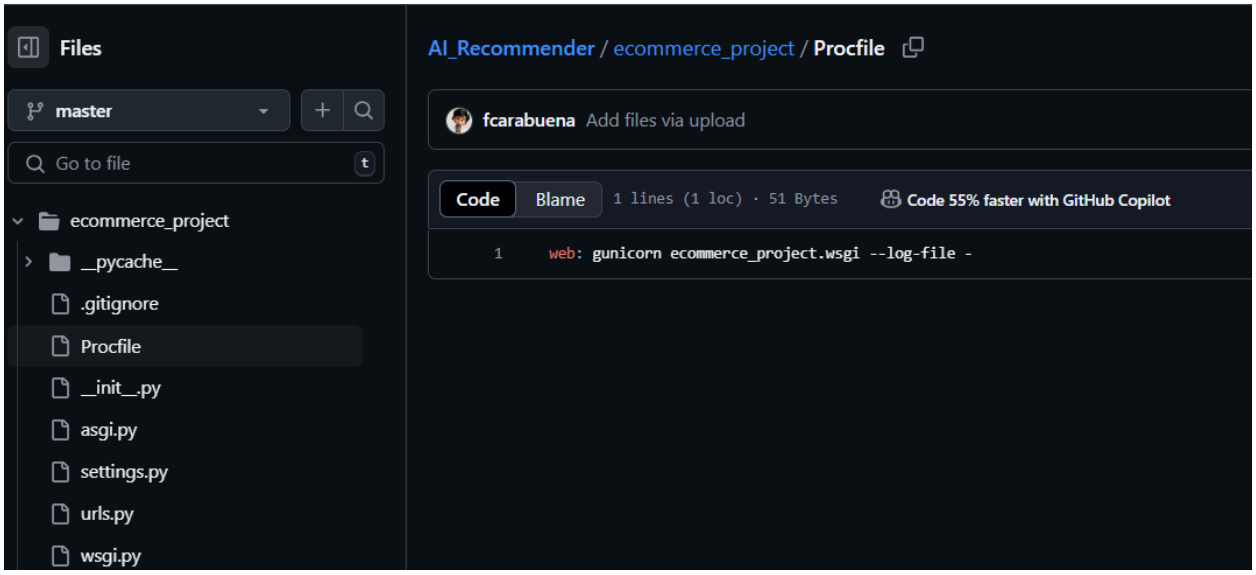
CONGRATULATIONS



Now Let's Review to Avoid Errors (You must have the following)


AI_Recommender
Public
Pin
Unwatch 1
master
1 Branch
0 Tags
Go to file
t
Add file
Code
fcarabuena Update settings.py
8a9d4cc · 7 minutes ago
16 Commits
ecommerce_project
Update settings.py
7 minutes ago
store
Initial commit: Set up Django project with MongoDB and AI ...
1 hour ago
manage.py
Initial commit: Set up Django project with MongoDB and AI ...
1 hour ago
requirements.txt
Update requirements.txt
25 minutes ago


AI_Recommender / requirements.txt
fcarabuena Update requirements.txt
Code
Blame
9 lines (9 loc) · 153 Bytes
Code 55% faster with GitHub Copilot
1 asgiref==3.8.1
2 Django==3.1.12
3 djongo==1.3.7
4 dnspython==2.7.0
5 pymongo==3.11.4
6 pytz==2025.2
7 sqlparse==0.2.4
8 typing_extensions==4.13.1
9 gunicorn



I hope nag RUN din without Errors :)

Challenge Task Recap (AI + Tags)

🔧 *Build a product tagging system that uses OpenAI to auto-suggest and store tags when a product is added.*

- Add a tags field to your model
- Generate tags from OpenAI in views.py
- Display them on product_list.html
- Save everything in MongoDB Atlas

Reflection Questions

In your Reflection of Learning, Answer the following:

1. How did version control help you manage changes during development?

2. What are the benefits of using cloud hosting like Render?

3. How does an AI recommender transform user experience in e-commerce?

4. Which part of the deployment process was the most challenging?

5. What improvements can be made if you plan to scale the application in production?